Efficient Type Inference for Elixir and Dynamically Typed Languages

PhD proposal for University Ca'Foscari, Computer Science department, Venice, Italy

Adrien A. R. Zinger

Introduction

Behind the scenes of programming languages, type systems constitute a theoretical fondation that every programmer encounters. They come in many forms: static or dynamic, strong or weak. With inference (such as `auto` in C++ and `let` in Rust), gradual in Python, nominal in C or structural typing as found in languages like Elixir with semantic subtyping. In all cases, programming languages strongly rely on type systems.

Although much has been written about whether static typing truly leads to better software, it is undeniable that systems offering robust type inference are widely appreciated, especially in interactive contexts. A natural question arises: why introduce static types into a language at all? Why not instead improve tooling, such as language server protocols, or *Dializer* for *Elixir*? The answer is not straightforward. (Neto, 2023)

We must mention that static types help to reduce the computation time of a static analyser. At some point, inferencing dynamically typed language are close to static analysis point. (Cousot, 1997)

The techniques used share a common drawback: they are often computationally expensive in dynamically typed languages. While such methods may be sufficient to catch errors before deployment, they are adapted for interactive use. On the other side, `bacon`, `gradle assemble -t` have tight feedback loop because of the statically typed context.

Topic definition

The central objective of this research is to study how type inference for gradually typed or partially annotated dynamically typed languages can be made efficient enough for interactive development.

The proposed approach combines an initial approximation phase, based on language-aware predicted annotations, with a second phase of formal refinement using techniques from type inference and abstract interpretation. The approximate information won't replace formal guarantees with heuristics, but guide the inference.

Elixir serves as the primary case study because of the quantity and the quality of research on semantic subtyping and gradual typing. Javascript is present in that paper because it surprisingly share some properties, it is also a good exercise to generalize the metodology. (Castagna et al., 2023)

Recent developments, such as *Luau*, which follows a trajectory similar to *Elixir* by incorporating annotated types, gradual typing, address this issue by initiating type inference through language-integrated mechanisms. It use of Large Language Models to bootstrap the inference process by generating initial partial annotations from unannotated code, thereby drastically reducing the search space. (Jeffrey, 2022)

This constitutes the first phase of the proposed project. The second phase consists of applying rigorous methods from type theory to refine and validate these inferred types.

The more pactical part, the implementation, prioritizes responsiveness, offering configurable trade-offs between precision and performance, and enabling optional code transformations such as type annotation insertion and safety fixes. The goal of this work is to design a toolchain that is fast enough to assist developers during active programming. It should not only be effective on high-performance machines but also usable on standard developer hardware.

Type inference remains inherently complex in codebases with limited type information, especially in the presence of polymorphism. Consequently, the system should provide users with mechanisms to configure analysis limits.

```
let a = null;
api.listen("/set", req => {
    a = req; // req is {val: string, ...}
});
api.listen("/get", req => {
    // required to be safe: if (a != null) {
        return a.val;
    // } else { return "a is null"; }
});
```

A code fragment such as the one shown above should trigger a warning for the developer. Indeed, there is no guarantee—outside of implicit business logic—that the `set` endpoint will be invoked before `get`. As a result, accessing `a` without a prior check can lead to an error.

The comments here are an example of what the tool may detect. It is not considered during analysis. In addition, such warning doesn't require to be aware of the which between `get` or `set` is called in a first place. Structural type systems collect all information about `a` such as its type is `null or map` with map an object description. At this point, any access to that variable is unsafe. In the next chapter we'll present how semantic subtyping handle automatically the convariance between `null or map` and `map`.

It is well established that expressive type systems, particularly when combined with semantic subtyping, provide significant benefits. Moreover, both static and gradual type systems enable static analyzers to detect bug patterns effectively. Modern statically typed languages such as *C++* and *Rust* further extend these benefits by integrating algebraic data types and structured forms of subtyping.

Historically, several dynamically typed languages have evolved by introducing gradual or static typing systems. Notable examples include *Elixir* in relation to *Erlang*, *TypeScript* for *JavaScript*, and *Luau* for *Lua*. While these evolutions provide significant benefits, they also require additional effort from developers, who must supply more type annotations or documentation.

The research area addressed by this proposal aims to detect such issues not only in gradually typed code but in general *Elixir* codebase which can be writen with no types's documentations at all.

Elixir would benefit from tools capable of automatically generating type annotations or code transformations for *Dialyzer*, in a manner similar to `cargo clippy --fix`, while leaving developers in control of whether to apply these suggestions. In addition, it should be able to improve code annotations (*Dializer* and *JSDoc* for instance).

Scalability Limits

Despite the extensive body of work on type inference and static analysis, existing approaches exhibit significant limitations when applied to modern dynamically typed languages. In particular, *Hindley–Milner–based* constraint resolution struggles to scale to features such as let-polymorphism. (Henglein & Mairson, 1994) Similarly, abstract interpretation techniques often face challenges in balancing performance and usability in interactive development settings.

As discussed in the introduction, statically typed languages are generally easier—though still non-trivial—to analyze and verify. In contrast, dynamically typed languages such as *Elixir*, *Python*, *Luau*, and *TypeScript* adopt partially similar strategies to mitigate this complexity.

In particular, they rely on two main mechanisms. First, they incorporate type annotations, or at least allow types to be expressed within the language syntax, thereby reducing the search space during inference. Second (for *Elixir*), they use guards to refine types within specific regions of code, for instance at the level of function definitions. (Castagna & Duboc, 2024)

This section presents some examples illustrating the limitations of type inference in the absence of sufficient type information, and how the techniques proposed in this work can help overcome these limitations.

Elixir currently employs a variation of *Hindley–Milner–based* inference, combining classical *Hindley–Milner* type inference with polymorphic types and union types. Comparable extensions of *Hindley–Milner* can be found in languages such as *Haskell* and *OCaml*, each incorporating their own adaptations.

```
f0 x = (x,x)
f1 x = f0 (f0 x)
f2 x = f1 (f1 x)
f3 x = f2 (f2 x)
f4 x = f3 (f3 x)
...
fn x = fn-1 (fn-1 x)
```

Inferring the types of such a program using a naive implementation of *Hindley–Milner* leads to a rapid growth in the size of generated types. At each application and let-binding, fresh type variables are introduced and duplicated, causing the inferred type structure to expand exponentially. As a result, the overall complexity of inference in this case is on the order of 2^n. (HackerNews, 2014)

Although this example is artificial and unlikely to appear directly in real-world codebases, similar patterns can arise in more practical settings. A classical illustration is the use of a fixpoint combinator:

```
let fixpoint = f => {
    let g = x => f(x(x));
    return g(g);
}

let res = fixpoint(fact)(n);
```

From the perspective of type inference, analyzing the fixpoint combinator involves reasoning about self-application, which can lead to recursive expansions analogous to repeatedly inferring expressions of the form `fact(fact(fact(...)))`. This produces a comparable blow-up in the size of inferred types and in the complexity of constraint resolution.

In practice, compilers for languages such as *OCaml* and *Haskell* mitigate these issues by recognizing common recursion patterns (e.g. `let rec`) and applying specialized inference strategies that ensure linear complexity. However, an implementation of `varargs` in *OCaml* like follow still can show a symptomatic long computation time. (HackerNews, 2014)

```
let sink (a,f) = f a
let base = ()
let finish () = ()
let step () = ()
let fold (a,f) g = g (a,f)
let step0 h (a,f) = fold (h a,f)
let f z = fold (base, finish) z
let a z = step0 step z
let () =
  let () = f
    a a a a
    a a a a
    a a a a
    a a a a
    a a a a
    a a a
    sink
  in
  ();;
```

Although type inference has been significantly optimized in some languages, it remains too costly in *Elixir* to support everyday interactive use. One important reason is that semantic subtyping is more difficult to manage than inference in a more classical nominal type system.

This additional complexity stems from at least two factors. First, semantic subtyping requires the system to retain substantially more information about type variables throughout the analysis. Second, the unification process (central to Hindley–Milner-style algorithms such as W and J) becomes more expensive when operating over richer

structural representations.

In some cases, semantic subtyping also requires forms of backtracking. These mechanisms make the analysis more expressive and more precise, but they also increase its computational cost.

This difficulty is central to the present research topic. However, explaining it directly in terms of constraint resolution can be unnecessarily technical at this stage. A more intuitive way to illustrate the issue is through the perspective of abstract interpretation.

```
foo(a) when isPositive(a) -> 1 / a;
foo(a) -> false;
```

In such fragment, a structural type system equipped with semantic subtyping can infer that `foo` has a type of the form

$$(\text{positive} \to \text{positive}) \vee (\neg\text{positive} \to : \text{false})$$

where `:false` denotes the atomic type corresponding to the value `false`. Both *Elixir*'s type system and an abstract interpreter can refine the type of `a` inside the first branch, concluding that `a` has the type positive in that context. The division is considered safe and should not trigger a warning. In the *Elixir* specification, this behavior is formalized in *Polymorphic Type Inference for Dynamic Languages* (Castagna et al., 2023), notably through the `[V]` rule.

Structural type systems permit also more complexe analysis like follow:

```
// bar is ({ty: "num", var: number} or {ty: "string", var: string})
if (bar.ty == "num") {
    // bar is just {ty: "num", var: number}
} else {
    // bar is just {ty: "string", var: string}
}
```

This illustrates one of the major strengths of structural typing: the ability to refine types by analyzing the shape and internal properties of values. Such precision goes beyond what is usually available in nominal type systems and is particularly valuable in dynamically typed languages.

However, this expressiveness comes at a cost. As the codebase grows, the space of possible refinements and intermediate type states may itself grow exponentially. This is why the exploration performed by the analysis must be bounded or guided. Optimizations such as memoization and iterative analysis can reduce computation time, but they do not appear sufficient.

For this reason, a deeper study of the properties of structural type systems, especially in the presence of polymorphism, is necessary. Such an investigation is essential to understand the computational impact of these techniques.

```
let v = init();
while (1) {
    v = modify()
}
```

In a fragment such as the one above, an abstract interpreter may fail to converge in some cases. A naive interpreter must analyze the body of the loop for every abstract state associated with `v`. Each time the abstract type of `v` changes, the interpreter must reconsider the assignment `v = modify()`. If this analysis keeps producing a new abstract type at every iteration, the process may continue indefinitely.

More precisely, the type of `v` may evolve toward a growing union of the form

$$T_1 \vee T_2 \vee \cdots \vee T_n$$

Termination can therefore only be guaranteed if this ascending sequence eventually stabilizes, that is, if the set of reachable abstract states remains finite.

In statically typed languages, this issue is often mitigated by explicit type information provided by the programmer. In particular, return types impose constraints on the result of a function and restrict the set of types that may appear during analysis. If an abstract interpreter for a dynamically typed language had access to comparable constraints, it could bound the exploration of the loop body more effectively and avoid repeatedly reanalyzing the same code under an ever-growing sequence of abstract states.

For example, in *Rust*, function signatures and algebraic data types provide such constraints directly:

```rust
// Traits abstraction
fn foo(x: impl Into<Bor>) -> Box<dyn Bar> {
    ...
}

// Enums
enum Bar {
    UsignedInt(u32),
    String(string),
    ...
}

fn foo(x: Bar) -> Bar {
   ...
}
```

In these examples, the declared input and output types restrict the range of possible values manipulated by the function. This does not eliminate the need for analysis, but it significantly limits the state space that must be explored. By contrast, in dynamically typed languages, where such bounds are often absent or only partially documented, the space may become immeasurable.

Hybrid Approach

The technical difficulties discussed in this proposal largely stem from the lack of explicit type information in dynamically typed languages. Yet these languages often remain more convenient to use than statically typed ones. For example, in Python, writing `vec[-1]` to access the last element of a list is perfectly natural. In *Rust*, by contrast, expressing the same operation is less straightforward because the compiler must account for boundary conditions, such as whether the vector may be empty at a given point in execution. (Matsakis, 2025)

A sufficiently expressive type system—particularly one equipped with semantic subtyping—could help bridge part of this gap between convenience and safety.

The first step of the proposed approach is therefore to use a trained model to annotate the abstract syntax tree of a dynamically or gradually typed program with likely type information. Once these annotations are available, functions such as function `modify()` can be assigned signatures that are directly exploitable by the type system. In the same vein, `fixpoint()` can be detected as a recursive function returning a number and close the problematics.

At this stage, the same information could also be used to generate annotations for the *Dialyzer*, thereby providing immediate practical value to developers.

This proposal is not intended to compete with profiling or deep diagnostic tool. Rather, its goal is to provide a typing mechanism that is lightweight enough to run on standard hardware while still being useful during development or to increase codebase quality. The output of this first phase would be an incomplete or approximate set of types associated with the user's code. Even approximate information—for example, knowledge that a value implements `toString`—can significantly reduce the search space explored by more rigorous inference methods. In this sense, the learned component is not a replacement for formal analysis, but a way to guide and accelerate it.

Formal Refinement

The output of the first phase is then processed by a more classical analysis framework, either based on *Hindley–Milner*-style type checking or on abstract interpretation. Part of this second phase may be language-independent, since several core principles of inference and refinement can be shared across ecosystems. However, a fully generic

solution is not realistic. Languages such as *Erlang* and *JavaScript* differ substantially in their models of concurrency, which means that some aspects of the analysis must remain language specific.

A central research question is therefore to characterize the properties of structural typing in such settings and to determine how these properties can be exploited efficiently in practice. We have which metrics should be collected to evaluate and diagnose the behavior of the tool. Given the wide adoption of languages such as *Elixir*, these ecosystems provide a strong basis for experimentation and for gathering meaningful feedback.

Methodology

Resume

This research is organized as a progressive study of type inference for dynamically and gradually typed languages, with a primary focus on *Elixir* and possible extensions to *JavaScript* and *Ruby*. The objective is to design a practical and formally grounded toolchain able to provide fast type information, warnings, and optional fixes during development.

Phase 1: Study of Existing Approaches

The first phase consists of reviewing existing work on *Hindley–Milner* inference, semantic subtyping, gradual typing, and abstract interpretation. The goal is to identify the theoretical and practical limits of current approaches when applied to dynamically typed languages. This phase establishes the foundations of the research and sharpens the problem statement.

Phase 2: Type Recovery and Formal Refinement

The second phase focuses on recovering missing type information directly from source code. A language-aware model will be used to annotate abstract syntax trees with approximate types. These annotations are intended to reduce the search space of later analyses and may also be used to generate optional type hints, such as *Elixir* type's annotations.

It also, at the end, refines the approximate types produced in the first stage using more classical methods, such as *Hindley–Milner*-style checking, abstract interpretation, or a combination of both. Some mechanisms may be shared across languages, while others must remain language-specific because of differences in semantics, concurrency, and state management. This phase connects approximate inference with formal reasoning.

Phase 3: Tooling Design

The third phase concerns the integration of these methods into a practical toolchain. The objective is to provide useful feedback on standard developer machines, including offline use. This phase also includes the design of configurable trade-offs between precision and response time, as well as the generation of optional warnings, fixes, and annotations.

Finally, that phase is also is devoted to evaluation on representative examples and real-world codebases. The system will be assessed in terms of precision, response time, scalability, and usefulness for developers. This phase will also measure how much the initial approximate typing reduces the cost of the later formal analysis.

Expected Contributions and Challenge

This work is expected to contribute both theoretically and practically. On the theoretical side, it should clarify the properties of current inference methods, especially in the semantic subtyping of *Elixir*. On the practical side, it aims to produce a prototype toolchain for developer assistance and a methodology for combining approximate and formal inference.

The main challenge is to balance precision and performance. A system that is to precise may be too slow, while a system that is too approximate may be of limited use. Other challenges include portability across languages and the definition of meaningful evaluation metrics.

Timetable

The research is expected to be structured over three years, with a progression from theoretical foundations to implementation and evaluation.

Year	Objectives	Expected outcomes
1	Conduct a detailed review of the literature on type inference, semantic subtyping, gradual typing, and abstract interpretation. Precisely define the research problem, delimit the scope of the thesis, and establish the theoretical foundations of the proposed approach.	State of the art, refined research questions, formalized scope, and initial methodological framework.
2	Develop the proposed hybrid approach by combining approximate type recovery with formal refinement methods. Study the extent to which the approach can be shared across languages while identifying the language-specific components required by different semantic models.	Prototype of the proposed framework, first experimental results, and intermediate validation of the methodology.
3	Consolidate the framework, extend its integration into developer-oriented tooling, and evaluate it on representative examples and real-world codebases. Finalize the theoretical analysis and synthesize the results into the dissertation.	Complete evaluation, validated contributions, identified limitations, and final manuscript.

References

Bernardy, J.-P., Boespfug, M., Newton, R. R., Jones, S. P., & Spiwack, A. (2018). Practical linearity in a higher-order polymorphic language. *Proceeding of the ACM Conference on Principles of Programming Languages*.

Castagna, G., & Duboc, G. (2024). Guard analysis and safe erasure gradual typing: A type system for elixir. *Institut de Recherche En Informatique Fondamentale, Université Paris Cité, CNRS, France*.

Castagna, G., Laurent, M., & Nguyên, K. (2023). Polymorphic type inference for dynamic languages. *Université Paris Cité, Université Paris-Saclay, CNRS, France*.

Cousot, P. (1997). Type as abstract interpretations. *LIENS, École Normale Supérieure, France*.

HackerNews. (2014). *Look at the humongous type that hindley-milner infers for this tiny program*. https://news.ycombinator.com/item?id=8130293.

Henglein, F., & Mairson, H. G. (1994). The complexity of type inference for higher-order typed lambda calculi. *Cambridge University Press*.

Jeffrey, A. (2022). Semantic subtyping in luau. *Roblox Technical Blog*.

Matsakis, N. (2025). *How i learned to stop worrying and love the LLM*. https://smallcultfollowing.com/babysteps/blog/2025/02/10/love-the-llm/.

Milner, R. (1977). A theory of type polymorphism in programming. *University of Edinburgh, Scotland*.

Neto, A. (2023). *Comments on static typing elixir*. https://dev.to/adolfont/static-typing-in-elixir-25p3.